

Universidad de los Andes

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

Métodos Estadísticos para la Gestión

Proyecto 2 — Informe 3

Integrantes

Agustín Álvarez

Arantxa Toledo

Samuel Vallete

Profesor

Jorge Gomez Miranda

Informe 3 — Assembler: codificación, etiquetas, validación y pruebas

Introducción

Este informe documenta el ensamblador propio del grupo (tools/assembler.py), una herramienta en Python desarrollada específicamente para este proyecto que traduce los programas en assembly RV32E (como sw/game.s) a archivos .hex cargables por el hardware. Se desarrolla por una restricción explícita del enunciado: el proyecto no puede depender de un ensamblador RISC-V externo (como riscv64-unknown-elf-as), por lo que todo el proceso de codificación a código máquina debe resolverse con herramientas propias.

El objetivo de este documento es describir en profundidad el funcionamiento interno del ensamblador: qué sintaxis acepta, cómo codifica cada instrucción soportada a su representación binaria de 32 bits, cómo resuelve etiquetas y constantes mediante un algoritmo de dos pasadas, qué errores detecta y cómo los reporta, y cómo se validó su correctitud mediante una suite de pruebas automatizadas.

Se cubren los siguientes aspectos: la sintaxis general aceptada por el ensamblador; la tabla completa de instrucciones soportadas y su codificación por formato (R, I, S, U, B, J); las pseudo-instrucciones y su expansión a instrucciones reales; el algoritmo de dos pasadas usado para resolver etiquetas; el tratamiento de inmediatos y de constantes definidas con .equ; las validaciones y los errores que el ensamblador es capaz de detectar; un ejemplo completo de traducción de assembly a código máquina; y, finalmente, un resumen de la suite de pruebas (tools/pruebas.py) y de sus resultados.

Sintaxis aceptada por el assembler

El ensamblador procesa el archivo de entrada línea por línea, con una sintaxis simple y permisiva en cuanto a formato (espacios, mayúsculas), pero estricta en cuanto a estructura. Cada línea puede contener, en cualquier combinación válida, una etiqueta, un mnemónico con sus operandos, un comentario, o estar vacía.

Estructura general de una línea

`[etiqueta:] [mnemónico [operando, operando, ...]] [# comentario]`

El parseo de cada línea (función analizar_linea) sigue estos pasos:

- Se descarta todo lo que sigue al carácter #, que marca el inicio de un comentario hasta el final de la línea.
- Si la línea contiene :, todo lo anterior al primer : se interpreta como una etiqueta (se

guarda en minúsculas tal como aparece, sin validación de formato adicional más allá de no estar duplicada).

- Lo que queda después de la etiqueta (si existe; si no, toda la línea) se separa en mnemónico y operandos: la primera palabra es el mnemónico (convertido a minúsculas, sin dependencia sobre la capitalización) y el resto de la línea se interpreta como una lista de operandos separados por comas.

Una línea puede contener solo una etiqueta (loop:), solo una instrucción sin etiqueta, una etiqueta seguida de instrucción en la misma línea (loop: addi x1, x0, 1), solo un comentario, o estar completamente vacía — todos estos casos son válidos y no generan código o etiquetas según corresponda.

Registros

El assembler reconoce únicamente los registros x0 a x15, consistente con el perfil RV32E implementado por el Espino Core. Los nombres de registro no distinguen mayúsculas de minúsculas, pero cualquier otro nombre, o un registro fuera de ese rango (x16 en adelante), es rechazado como error.

Operandos de memoria

Para las instrucciones de carga y almacenamiento (lw, sw), el operando de memoria sigue la sintaxis offset(registro), por ejemplo 4(x2) o -4(x2). El offset es opcional y, si se omite, se asume 0 ((x2) equivale a 0(x2)); el offset puede expresarse en decimal o en hexadecimal con prefijo 0x (por ejemplo 0x10(x5)), siguiendo las reglas estándar de interpretación numérica de Python (int(texto, 0)).

Inmediatos

Los valores inmediatos (en instrucciones tipo I, U, y en el offset de memoria) se interpretan también en formato decimal o hexadecimal con prefijo 0x, y se validan contra el rango permitido por el formato de instrucción correspondiente (ver “Tratamiento de inmediatos y constantes”).

Directivas

La única directiva reconocida por el ensamblador es .equ NOMBRE, valor, usada para definir constantes simbólicas. Cualquier mnemónico que comience con . se trata como directiva y no genera código máquina directamente; el resto de la línea, en el caso de .equ, se interpreta como el nombre de la constante y su valor.

Comentarios y líneas vacías

Los comentarios comienzan con # y se extienden hasta el final de la línea, pudiendo aparecer solos (comentario de línea completa) o al final de una instrucción. Las líneas vacías o compuestas solo por espacios en blanco se ignoran completamente, sin generar error ni ocupar espacio en el programa.

Instrucciones soportadas y su codificación

El ensamblador codifica cada instrucción según su formato (R, I, S, U, B o J), definidos todos en la tabla TABLA del código fuente, que asocia cada mnemónico con su formato, su opcode, y (cuando corresponde) su funct3 y funct7. La siguiente tabla resume las 19 instrucciones reales soportadas:

Mnemónico	Formato	Opcode	funct3	funct7
add	R	0x33	0	0x00
sub	R	0x33	0	0x20
xor	R	0x33	4	0x00
or	R	0x33	6	0x00
and	R	0x33	7	0x00
sltu	R	0x33	3	0x00
addi	I	0x13	0	—
andi	I	0x13	7	—
xori	I	0x13	4	—
ori	I	0x13	6	—
lui	U	0x37	—	—
lw	L (I)	0x03	2	—
sw	S	0x23	2	—

Mnemónico	Formato	Opcode	funct3	funct7
beq	B	0x63	0	—
bne	B	0x63	1	—
blt	B	0x63	4	—
bge	B	0x63	5	—
bltu	B	0x63	6	—
bgeu	B	0x63	7	—
jal	J	0x6F	—	—

El formato “L” es una variante propia del assembler para distinguir la carga de memoria (lw) del resto de instrucciones tipo I aritméticas, ya que ambas comparten la misma codificación de campos (tipo I estándar de RISC-V) pero difieren en cómo se interpretan sus operandos de texto (registro + inmediato vs. registro + operando de memoria).

Codificación por formato

Cada formato se codifica mediante una función dedicada que arma la palabra de 32 bits combinando sus campos con desplazamientos de bits y máscaras, siguiendo la especificación estándar de codificación de instrucciones RISC-V:

- **Tipo R (enc_r):** combina funct7, rs2, rs1, funct3, rd y opcode — para operaciones registro-registro como add, sub, xor, or, and, sltu.
- **Tipo I (enc_i):** combina un inmediato de 12 bits, rs1, funct3, rd y opcode — para operaciones registro-inmediato como addi, andi, xori, ori, y también para lw (reutilizando esta misma función, ya que lw es, en términos de codificación binaria, una instrucción tipo I).
- **Tipo S (enc_s):** separa el inmediato de 12 bits en dos partes (bits altos y bajos) para codificar alrededor de los campos rs1 y rs2 — usado exclusivamente por sw.
- **Tipo U (enc_u):** coloca un inmediato de 20 bits en la mitad alta de la palabra — usado por lui, la única instrucción de este tipo soportada.
- **Tipo B (enc_b):** codifica un offset de salto condicional de 13 bits (siempre par, el bit 0 se omite) distribuido en varios campos non contiguos — usado por las seis

instrucciones de branch (beq, bne, blt, bge, bltu, bgeu).

- **Tipo J (enc_j):** codifica un offset de salto incondicional de 21 bits (también siempre par) distribuido de forma no contigua — usado exclusivamente por jal.

Esta correspondencia exacta entre mnemónico, formato y campos de codificación está validada directamente por la suite de pruebas (tools/pruebas.py), que verifica para cada función de codificación que un conjunto de entradas conocidas produzca exactamente el valor hexadecimal esperado.

Pseudo-instrucciones y su expansión

Además de las 19 instrucciones reales de la tabla anterior, el ensamblador reconoce un conjunto de pseudo-instrucciones: mnemónicos que no existen como tales en el hardware, pero que se traducen (se "expanden") automáticamente a una o más instrucciones reales antes de codificarse. Esta expansión ocurre en la función expandir, que se invoca tanto en la primera pasada (para calcular correctamente las direcciones de las etiquetas, ver siguiente sección) como en la segunda pasada (para generar el código final).

Pseudo-instrucciones simples

Pseudo-instrucción	Operandos	Se expande a
nop	(ninguno)	addi x0, x0, 0
mv rd, rs	2	addi rd, rs, 0
j etiqueta	1	jal x0, etiqueta
beqz rs, etiqueta	2	beq rs, x0, etiqueta
bnez rs, etiqueta	2	bne rs, x0, etiqueta

Cada pseudo-instrucción se traduce a solamente una instrucción real, reutilizando x0 (que en RISC-V siempre vale 0) como operando implícito. mv se convierte en una suma con 0; j es un salto incondicional implementado como jal con registro de retorno descartado (x0); beqz/bnez son comparaciones contra 0 usando beq/bne.

Expansión de li (load immediate)

La pseudo-instrucción li rd, valor es la única cuya expansión es condicional: puede generar una o duas instrucciones reales, dependiendo de la magnitud del valor a cargar.

```
if -2048 <= valor <= 2047:
    return [("addi", [rd, "x0", str(valor)])]
bajo = valor & 0xFFF
if bajo >= 2048:
    bajo -= 4096
alto = ((valor - bajo) >> 12) & 0xFFFFF
if bajo == 0:
    return [("lui", [rd, str(alto)])]
return [("lui", [rd, str(alto)]), ("addi", [rd, rd, str(bajo)])]
```

El criterio es el siguiente:

- Si el valor cabe en el rango de un inmediato de 12 bits con signo (−2048 a 2047), basta con una sola instrucción addi rd, x0, valor.
- Si el valor es mayor, se descompone en dos partes: los 12 bits bajos (bajo), ajustados para quedar en el rango con signo (−2048 a 2047, restando 4096 si es necesario), y los 20 bits altos restantes (alto), calculados de forma que lui rd, alto seguido de addi rd, rd, bajo reconstruya exactamente el valor original.
- Como caso particular, si la parte baja resulta ser exactamente 0, se omite la instrucción addi y solo se emite lui, ahorrando una palabra de código.

Esta pseudo-instrucción es la que permite cargar las constantes de temporización del juego (TENTH = 2.500.000, TRES_S = 75.000.000), que exceden ampliamente el rango de 12 bits y requieren la expansión de dos instrucciones. Por ejemplo, li x5, 2500000 se expande a lui x5, 610 seguido de addi x5, x5, 1440 — valores verificados directamente en la suite de pruebas.

Validación de cantidad de operandos en pseudo-instrucciones

Antes de aplicar cualquier expansión, el ensamblador valida que la pseudo-instrucción haya recibido la cantidad exacta de operandos esperada (tabla PSEUDO_OPERANDOS: 0 para nop, 2 para mv/beqz/bnez/li, 1 para j), rechazando con un error explícito cualquier desviación.

Impacto en el cálculo de direcciones

Un detalle importante es que la expansión de li en dos instrucciones ocupa dos palabras de memoria (8 bytes) en lugar de una, lo que afecta directamente el cálculo de direcciones de las etiquetas que aparecen después de un li de constante grande en el código. El ensamblador resuelve esto correctamente porque la primera pasada (que calcula las direcciones) también invoca expandir para contar cuántas palabras genera cada línea, en lugar de asumir que cada línea de assembly ocupa siempre una única instrucción (ver siguiente sección, “Tratamiento de etiquetas”).

Tratamiento de etiquetas

El ensamblador resuelve las etiquetas del programa mediante un algoritmo clásico de dos pasadas, necesario porque una instrucción de salto (beq, jal, o sus pseudo-instrucciones j,

beqz, bnez) puede referirse a una etiqueta que aparece más adelante en el código (salto hacia adelante), cuya dirección todavía no se conoce en el momento de procesar esa línea.

Primera pasada: construcción de la tabla de etiquetas

```
def primera_pasada(lineas, constantes={}):
    etiquetas = {}
    pc = 0
    for n, linea in enumerate(lineas, 1):
        etiqueta, mnemonico, operandos = analizar_linea(linea)
        if etiqueta is not None:
            if etiqueta in etiquetas:
                raise ValueError(f"linea {n}: etiqueta duplicada '{etiqueta}'")
            etiquetas[etiqueta] = pc
        if mnemonico is None or mnemonico.startswith("."):
            continue
        operandos = sustituir_constantes(operandos, constantes)
        try:
            cantidad = len(expandir(mnemonico, operandos))
        except ValueError:
            cantidad = 1
        pc += 4 * cantidad
    return etiquetas
```

La función recorre el programa línea por línea, manteniendo un contador de dirección (pc, program counter) que comienza en 0 y avanza en 4 bytes por cada instrucción real generada. Cuando una línea define una etiqueta, esta se guarda en el diccionario etiquetas con el valor actual de pc — es decir, la dirección de la siguiente instrucción, no de la etiqueta en sí misma. Si la misma etiqueta aparece más de una vez en el programa, se reporta como error inmediatamente (“etiqueta duplicada”).

Un punto importante de esta implementación es que, para avanzar el contador pc, la primera pasada invoca la misma función expandir usada para generar el código final (ver sección anterior), en lugar de asumir que cada línea genera exactamente una palabra. Esto es lo que permite que el conteo de direcciones sea correcto incluso cuando hay pseudo-instrucciones que se expanden a dos instrucciones reales, como li con una constante grande: si no se contara correctamente el espacio ocupado por esos dos instrucciones, todas las etiquetas posteriores en el programa quedarían con una dirección incorrecta, y cualquier salto hacia ellas (o desde ellas) calcularía un offset erróneo.

Segunda pasada: codificación usando la tabla de etiquetas

Con la tabla de etiquetas ya completa, la segunda pasada recorre el programa nuevamente, esta vez generando el código máquina real de cada instrucción. En este punto, cualquier instrucción de salto puede resolver su etiqueta de destino consultando directamente el diccionario etiquetas, sin importar si esa etiqueta aparece antes o después en el código fuente.

Cálculo del offset de salto

```
def calcular_offset(destino, pc, etiquetas, minimo=-4096, maximo=4094):
    if destino not in etiquetas:
        raise ValueError(f"etiqueta no definida: {destino}")
    offset = etiquetas[destino] - pc
    if not (minimo <= offset <= maximo):
        raise ValueError(f"salto demasiado lejos: {offset} bytes (permitido {minimo} a
{maximo})")
    return offset
```

Para las instrucciones tipo B (branches) y tipo J (jal), el operando no es un registro ni un inmediato literal, sino el nombre de una etiqueta. El ensamblador calcula el offset de salto como la diferencia entre la dirección de destino (etiquetas[destino]) y la dirección de la instrucción de salto misma (pc). Este offset se valida contra el rango permitido por el formato: ±4096 bytes para saltos tipo B (limitado por los 13 bits con signo del campo de offset, siempre par), y un rango mucho mayor (±~1 MB) para saltos tipo J (jal), usado internamente por calcular_offset con límites distintos según el llamador. Si la etiqueta referenciada no existe en la tabla, o si el salto excede el rango representable, el ensamblador reporta el error correspondiente (“etiqueta no definida” o “salto demasiado lejos”).

Tratamiento de inmediatos y constantes (.equ)

Validación de inmediatos

Todo valor inmediato que aparece en el código fuente —como podrían ser una instrucción tipo I un inmediato de lui o el offset de un operando de memoria— se interpreta y valida mediante la función parse_imm:

```
def parse_imm(texto, minimo, maximo):
    try:
        valor = int(texto, 0)
    except ValueError:
        raise ValueError(f"inmediato invalido: '{texto}'")
    if not (minimo <= valor <= maximo):
        raise ValueError(f"inmediato fuera de rango: {valor} (permitido {minimo} a {maximo})")
    return valor
```

El texto se convierte a entero usando la función int de Python con base 0, lo que permite interpretar tanto notación decimal (100, -5) como hexadecimal con prefijo 0x (0x10), sin necesidad de lógica adicional. Si el texto no corresponde a un número válido, se reporta “inmediato invalido”. El rango permitido depende del contexto de uso: −2048 a 2047 para inmediatos de 12 bits con signo (instrucciones tipo I y offsets de memoria), y 0 a 0xFFFFFF (20 bits sin signo) para el inmediato de lui. Si el valor está fuera del rango correspondiente, se reporta “inmediato fuera de rango”, indicando además el rango permitido y el valor recibido, para facilitar la depuración.

Constantes simbólicas con .equ

El ensamblador permite definir constantes con nombre mediante la directiva .equ NOMBRE, valor, procesada antes de cualquier otra pasada del archivo:

```
def leer_constantes(lineas):
    constantes = {}
    for n, linea in enumerate(lineas, 1):
        etiqueta, mnemonico, operandos = analizar_linea(linea)
        if mnemonico == ".equ":
            if len(operandos) != 2:
                raise ValueError(f"linea {n}: .equ necesita: .equ NOMBRE, valor")
            if operandos[0] in constantes:
                raise ValueError(f"linea {n}: constante duplicada '{operandos[0]}'")
            valor = sustituir_constantes([operandos[1]], constantes)[0]
            try:
                constantes[operandos[0]] = int(valor, 0)
            except ValueError:
                raise ValueError(f"linea {n}: valor invalido en .equ: '{operandos[1]}'")
    return constantes
```

Esta función se ejecuta en una pasada previa, independiente de la que calcula las etiquetas, y recorre todo el archivo buscando directivas .equ. Cada constante se guarda con su nombre y su valor entero en un diccionario (constantes), validando que no exista una constante con el mismo nombre ya definida (“constante duplicada”) y que el valor asignado sea un número válido (“valor invalido en .equ”). Un detalle relevante es que el valor de una nueva constante puede referirse a una constante ya definida anteriormente (por ejemplo, .equ B, A), gracias a que sustituir_constantes se aplica también sobre el valor antes de convertirlo a entero — es decir, las constantes se resuelven en el mismo orden en que aparecen en el archivo, permitiendo composición simple entre ellas.

Sustitución de constantes en los operandos

Una vez construido el diccionario de constantes, cada línea del programa pasa por sustituir_constantes antes de ser ensamblada, tanto en la primera pasada (para el cálculo de direcciones) como en la segunda (para la codificación final):

```
def sustituir_constantes(operandos, constantes):
    nuevos = []
    for op in operandos:
        if "(" in op:
            antes, resto = op.split("(", 1)
            antes = antes.strip()
            if antes in constantes:
                antes = str(constantes[antes])
            op = antes + "(" + resto
        elif op in constantes:
```

```
            op = str(constantes[op])
        nuevos.append(op)
    return nuevos
```

Esta función recorre cada operando de la instrucción y, si coincide exactamente con el nombre de una constante conocida, lo reemplaza por su valor numérico (como texto). El caso especial es el de los operandos de memoria: si el operando contiene un paréntesis (por ejemplo OFFLED(x2)), solo se sustituye la parte anterior al paréntesis (el offset), dejando intacto el registro entre paréntesis — permitiendo así expresiones como sw x6, OFFLED(x2) donde OFFLED es una constante definida con .equ. Este mecanismo es el que permite que game.s use nombres descriptivos como DIGITOS, LEDS, BOTONES, CICLOS, TENTH, TRES_S y DIEZ en lugar de valores numéricos repetidos por todo el código, mejorando la legibilidad sin perder la capacidad de validación de rangos (la sustitución ocurre antes de parse_imm, por lo que una constante con un valor fuera de rango se detecta igual que un inmediato literal fuera de rango).

Validaciones y manejo de errores

El ensamblador fue diseñado para no detenerse ante el primer error encontrado, sino para recorrer todo el archivo y reportar de una sola vez todos los problemas detectados, facilitando la depuración de programas con múltiples errores. Esto se logra mediante la función ensamblar_archivo, que envuelve cada etapa del procesamiento en bloques try/except que acumulan los mensajes de error en una lista (errores) en lugar de interrumpir la ejecución inmediatamente.

Estrategia de acumulación de errores

El procesamiento de un archivo completo sigue estas etapas, cada una tolerante a fallos parciales:

- Lectura de constantes (.equ): si falla, se continúa con un diccionario de constantes vacío, para no bloquear el resto del análisis.
- Primera pasada tolerante: se recorre el archivo calculando direcciones y etiquetas línea por línea; si una línea no puede expandirse correctamente (por ejemplo, por un error de sintaxis), se asume que ocupa una sola palabra para no desalinear el conteo del resto del programa, y el error real se reportará en la segunda pasada.
- Segunda pasada: cada línea se ensambla individualmente dentro de su propio try/except; si falla, se agrega el mensaje de error (con el número de línea) a la lista de errores y se continúa con la siguiente línea, en lugar de abortar todo el proceso.
- Validación de tamaño del programa: al final, se verifica que el total de palabras generadas no exceda el límite de la RAM del SoC.

Si la lista de errores no está vacía al terminar, se lanza una única excepción con todos los mensajes concatenados (uno por línea); el archivo .hex de salida solo se escribe si no hubo ningún error.

Este diseño es el que permite que, por ejemplo, un archivo con errores en las líneas 1, 2 y 3

reporte los tres simultáneamente, en lugar de obligar a corregir uno, volver a ejecutar el ensamblador, encontrar el siguiente, y así sucesivamente.

Errores detectados

El ensamblador detecta y reporta, con mensajes descriptivos y (cuando corresponde) el número de línea exacto, los siguientes casos:

Error	Causa	Ejemplo
instruccion desconocida	Mnemónico no existe en la tabla de instrucciones ni como pseudo-instrucción	mul x1, x2, x3
registro invalido	Registro fuera del rango x0–x15, o nombre no reconocido	addi x16, x0, 1 / addi foo, x0, 1
espera N operandos, recibio M	Cantidad incorrecta de operandos para el mnemónico (real o pseudo)	add x1, x2 (faltan) / addi x1, x0, 1, 2 (sobran)
inmediato invalido	El texto del inmediato no es un número interpretable	addi x1, x0, hola
inmediato fuera de rango	El inmediato excede el rango permitido por el formato	addi x1, x0, 2048 / lui x1, 0x100000
operando de memoria invalido	Falta la sintaxis offset(registro) en lw/sw	lw x1, 4
etiqueta no definida	Un salto referencia una etiqueta que no existe en el programa	jal x0, nada
etiqueta duplicada	La misma etiqueta se define más de una vez	loop: ... loop: ...

Error	Causa	Ejemplo
salto demasiado lejos	El offset calculado excede el rango representable (B o J)	un beq hacia una etiqueta a más de 4096 bytes
constante duplicada	La misma constante .equ se define más de una vez	.equ A, 1 / .equ A, 2
valor invalido en .equ	El valor asignado a una constante no es un número	.equ A, zzz
programa demasiado grande	El total de palabras generadas supera el límite de 512 (tamaño de la RAM)	un programa con más de 512 instrucciones reales

Validación del tamaño máximo del programa

```
MAX_PALABRAS = 512
...
if len(palabras) > MAX_PALABRAS:
    errores.append(f"programa demasiado grande: {len(palabras)} palabras "
                   f"(maximo {MAX_PALABRAS})")
```

Este límite corresponde a el parámetro NumWords de pochoco_ram en el RTL del SoC, es decir, el tamaño real de la memoria del hardware objetivo. Validar este límite en tiempo de ensamblado evita que un programa demasiado grande se cargue de forma silenciosamente truncada o corrupta en la RAM al sintetizar el bitstream, detectando el problema en una etapa mucho más temprana y económica del flujo de desarrollo.

Ejemplo de traducción assembly → código máquina

A continuación se desarrolla, paso a paso, la traducción completa de dos instrucciones reales tomadas de la suite de pruebas del ensamblador, mostrando explícitamente cómo el texto en assembly se convierte en los campos binarios de la instrucción y, finalmente, en el valor hexadecimal de 32 bits.

Ejemplo 1: instrucción tipo R — sub x5, x3, x4

Esta instrucción resta el contenido de x4 al de x3 y guarda el resultado en x5. Según la tabla de instrucciones (ver “Instrucciones soportadas y su codificación”), sub es tipo R, con opcode 0x33, funct3 0, funct7 0x20. Los operandos se interpretan como rd = x5, rs1 = x3,

rs2 = x4.

La función enc_r arma la palabra de 32 bits concatenando los campos en el orden que exige la codificación RISC-V tipo R:

Campo	funct7	rs2	rs1	funct3	rd	opcode
Bits	31–25	24–20	19–15	14–12	11–7	6–0
Valor	0x20 (0100000)	4 (00100)	3 (00011)	0 (000)	5 (00101)	0x33 (0110011)

Concatenando estos campos de izquierda a derecha se obtiene la palabra binaria completa:

`0100000 00100 00011 000 00101 0110011`

Que, agrupada en bytes y expresada en hexadecimal, da exactamente:

`404182b3`

Este resultado está verificado directamente en pruebas.py (“enc_r sub x5, x3, x4” → "404182b3").

Ejemplo 2: instrucción tipo B con etiqueta — bne x1, x0, loop

Este ejemplo muestra el flujo completo, incluyendo la resolución de la etiqueta. Supongamos el siguiente fragmento, tomado también de la suite de pruebas, con la tabla de etiquetas ya calculada por la primera pasada:

```
loop: ...        # dirección de 'loop' = 4
...
    bne x1, x0, loop   # esta instrucción está en pc = 8
```

Paso 1 — Cálculo del offset de salto: calcular_offset("loop", pc=8, etiquetas={"loop": 4, ...}) calcula offset = etiquetas["loop"] - pc = 4 - 8 = -4 bytes (un salto hacia atrás de una instrucción). El valor cae dentro del rango permitido (−4096 a 4094), por lo que se acepta.

Paso 2 — Codificación con enc_b: bne es tipo B, con opcode 0x63 y funct3 1. Los operandos son rs1 = x1, rs2 = x0, imm = -4. La función enc_b distribuye los 13 bits del offset de forma no contigua entre varios campos de la palabra (bit 12 en el bit 31, bits 10–5 en los bits 30–25, rs2 en 24–20, rs1 en 19–15, funct3 en 14–12, bits 4–1 en 11–8, bit 11 en el bit 7, y el opcode en los bits 6–0) — una codificación diseñada en RISC-V para simplificar la generación del offset en hardware, a costa de no ser un campo contiguo.

Paso 3 — Resultado: aplicando esta distribución de bits al offset -4 (en complemento a dos de 13 bits: 1111111111100), junto con rs1=1, rs2=0, funct3=1 y opcode=0x63, se obtiene:

`fe009ee3`

Resultado también verificado en pruebas.py (“bne x1, x0, loop (pc=8, hacia atras)” → "fe009ee3").

Qué demuestran estos dos ejemplos

El primer ejemplo (tipo R) ilustra la codificación directa de campos fijos — sin dependencias externas al conjunto de operandos de la propia instrucción. El segundo (tipo B) ilustra el caso más complejo: una codificación que depende del resultado de una etapa previa del ensamblador (la tabla de etiquetas construida en la primera pasada) y que distribuye sus bits de forma non trivial, mostrando de manera concreta por qué el algoritmo de dos pasadas es imprescindible para resolver correctamente los saltos del programa.

Pruebas realizadas

La corrección del ensamblador se valida mediante tools/pruebas.py, una suite de pruebas propia (sin depender de un framework externo como pytest) que ejecuta 107 verificaciones individuales, organizadas en las siguientes categorías:

- **Encoders:** Verifica cada función de codificación (enc_r, enc_u, enc_s, enc_i, enc_j, enc_b) de forma aislada, comparando su salida contra valores hexadecimales calculados a mano para casos representativos de cada formato — incluyendo casos con inmediatos negativos (como sw x1, -4(x2), addi x3, x3, -1) y saltos tanto hacia adelante como hacia atrás (como jal x0, -8, jal x0, +8).
- **Funciones auxiliares:** Prueba las funciones de parseo de bajo nivel de forma independiente: parse_reg (incluyendo manejo de mayúsculas), parse_mem (con y sin offset, en decimal y hexadecimal), y analizar_linea (separación de etiqueta, mnemónico y operandos en distintas combinaciones, incluyendo comentarios y líneas vacías).
- **Instrucciones completas:** Ensambla directamente fragmentos de assembly de una sola línea (las 19 instrucciones reales de la tabla) y compara el resultado contra el valor hexadecimal esperado, incluyendo casos con mayúsculas y espacios irregulares, y con etiquetas en la misma línea que la instrucción.
- **Saltos con etiquetas:** Valida específicamente la codificación de bne y jal cuando referencian una tabla de etiquetas ya construida, cubriendo tanto saltos hacia atrás como hacia adelante.
- **Tabla de etiquetas (primera pasada):** Ejecuta primera_pasada sobre los tres programas de ejemplo que acompañan al repositorio (buttons_leds.s, blink.s, 7seg.s) y compara las direcciones calculadas para cada etiqueta contra los valores esperados, validando el algoritmo de conteo de direcciones sobre código real, no solo sobre fragmentos sintéticos.
- **Pseudo-instrucciones:** Verifica la expansión de nop, mv, j, beqz, bnez y li (incluyendo los distintos casos de li: inmediato pequeño, inmediato grande con lui+addi, caso donde la parte baja es exactamente 0, e inmediatos negativos como -1 o 0xFFFFFFFF), tanto a nivel de expansión simbólica como de código máquina final.
- **Programas oficiales del repo:** Ensambla completos los tres programas de ejemplo

(buttons_leds.s, blink.s, 7seg.s) y compara el .hex generado, línea por línea, contra el .hex oficial ya presente en el repositorio. Esta es una prueba de regresión de extremo a extremo que confirma que el ensamblador reproduce exactamente el comportamiento esperado sobre programas reales, no solo sobre casos de prueba aislados.

- **Constantes .equ:** Valida la sustitución de constantes en distintos contextos (operando simple, offset de memoria, uso de una constante dentro de la definición de otra) y la lectura de múltiples .equ en un mismo archivo.
- **Archivos completos con constantes y li:** Ensambla programas completos de varias líneas que combinan .equ, li y etiquetas, verificando que la interacción entre estos tres mecanismos —particularmente el cálculo de direcciones cuando hay instrucciones li de dos palabras— produzca el código máquina correcto.
- **Errores que deben detectarse:** Cubre sistemáticamente cada uno de los errores listados en la sección anterior (“Validaciones y manejo de errores”), verificando no solo que se lance una excepción, sino que el mensaje contenga el texto esperado (por ejemplo, que un registro inválido efectivamente diga “registro invalido” y no otro mensaje genérico).
- **Errores en archivos completos:** Verifica el comportamiento de acumulación de errores a nivel de archivo completo: que el número de línea reportado sea el correcto, que múltiples errores en el mismo archivo se informen todos a la vez, y casos límite como un programa de exactamente 600 instrucciones NOP (que excede las 512 palabras de RAM) o un salto que queda fuera de rango dentro de un archivo de cientos de líneas.

Resultado de la ejecución

Se ejecutó la suite completa (python3 tools/pruebas.py) directamente sobre el repositorio clonado, confirmando que las 107 pruebas pasan sin fallas:

```
=== 107 de 107 pruebas pasaron ===
```

Este resultado incluye, en particular, la verificación de regresión contra los tres .hex oficiales del repositorio, lo que da una garantía razonable de que el ensamblador no introduce errores de codificación en programas assembly reales, no solo en los casos de prueba sintéticos diseñados para cubrir cada regla individualmente.

Conclusión

El ensamblador propio del grupo (tools/assembler.py) incluye todo el proceso necesario para traducir un programa en assembly RV32E a código máquina ejecutable por el Espino Core en poco más de 300 líneas: codificación de los seis formatos de instrucción de RISC-V (R, I, S, U, B, J), expansión de siete pseudo-instrucciones de uso común, resolución de etiquetas mediante un algoritmo estándar de dos pasadas, soporte de constantes simbólicas, y una validación exhaustiva que cubre tanto errores de sintaxis como restricciones específicas del hardware objetivo (16 registros por RV32E, 512 palabras de RAM).

El diseño prioriza la utilidad práctica para el desarrollo del juego: los errores se acumulan y reportan todos juntos con su número de línea, en lugar de detenerse en el primero encontrado, lo que acelera la iteración al escribir y depurar assembly. Esta decisión de diseño, junto con el resto de la lógica del ensamblador, está respaldada por una suite de 107 pruebas automatizadas que se ejecutan y pasan en su totalidad, incluyendo una verificación de regresión de extremo a extremo contra los tres programas de ejemplo oficiales del repositorio en vez de comparar con los mismos casos de prueba estáticos.

En conjunto, el ensamblador demuestra ser una herramienta confiable y suficientemente completa para las necesidades del proyecto: fue la pieza que permitió traducir game.s a game.hex, el archivo efectivamente cargado por la RAM del Pochoco SoC, cerrando así el flujo completo desde el diseño de hardware hasta el programa ejecutándose en la FPGA.